

VFLUX 2 Documentation

Version 2.0.0

March 16, 2015

VFLUX 2: Vertical Fluid Heat Transport Solver (VFlu[H]X Solver)

1. Description

VFLUX is a program that calculates one-dimensional vertical fluid flow (seepage flux) through saturated porous media, using heat transport equations developed by Hatch et al. (2006) and Keery et al. (2007). It uses temperature time series data measured by multiple temperature sensors in a vertical profile in order to calculate flux at specific times and depths (Figure 1).

VFLUX 2 builds upon the initial VFLUX release, including heat transport equations developed by McCallum et al. (2012) and Luce et al. (2013). These new approaches calculate vertical fluid flow (seepage flux), as well as thermal diffusivity, and in the case of Luce et al. (2013), sensor spacing which can be used to determine whether streambed scour has occurred. No additional user input is required to run the main VFLUX 2 programs relative to previous releases.

VFLUX processes the temperature time series by first resampling the data to a lower sampling rate, in order to reduce the filtering problems associated with oversampling. The program then filters each time series using Dynamic Harmonic Regression (DHR) programs from the Captain Toolbox (Young et al. 1999; Young et al. 2010), isolating a fundamental temperature signal identified by its period of oscillation (typically a diurnal signal). Finally, the program calculates vertical flux, thermal diffusivity and sensor spacing between pairs of temperature sensors using the amplitudes and phases of their filtered temperature signals, according to Hatch et al. (2006) Keery et al. (2007), McCallum et al. (2012) and Luce et al. (2013). Please see Gordon et al. (2012) for more details of the VFLUX method and examples of its use.

VFLUX is distributed as a MATLAB toolbox, a set of functions written in the MATLAB computing language that are designed to run in the MATLAB environment. It was originally developed in MATLAB 7.10.0 (R2010a). A working familiarity with MATLAB is needed to use VFLUX. VFLUX requires the 'resample' function from the MATLAB Signal Processing Toolbox and the DHR functions from the Captain Toolbox (<http://www.es.lancs.ac.uk/cres/captain>). If the Signal Processing Toolbox is not available, then an alternate 'resample' function may be used (see Installation, below).

2. Installation

The VFLUX program can be downloaded from the following web site: <http://hydrology.syr.edu/vflux>. Copy the VFLUX toolbox folder (“vflux”) into a folder on the MATLAB search path, such as “My Documents\MATLAB” in Windows environments. Alternately, the location of the vflux folder can be added to the search path by typing `addpath('C:\examplefolder\vflux')` at the MATLAB command prompt. The MATLAB Signal Processing Toolbox must be installed, and the Captain Toolbox folder (“captain”) must also be added to the MATLAB search path. The Captain Toolbox can be downloaded from the following web site: <http://www.es.lancs.ac.uk/cres/captain>.

If the Signal Processing Toolbox is not available, then a private ‘resample’ function may be used. This function can be downloaded along with VFLUX from the above web site, and must be copied to a folder called “private” within the VFLUX toolbox folder (“vflux”). The MATLAB Compiler Runtime (MCR) may also need to be installed; see the readme file included with the private resample function for more information.

There are two functions in the toolbox that can be directly called by the user, `VFLUXFormat` (`vfluxformat`) and `VFLUX` (`vflux`). These functions are executed by typing their commands at the MATLAB command prompt, and can also be called by external functions or scripts, allowing them to be integrated into your custom workflow. `VFLUXFormat` is used to format and (optionally) synchronize multiple time series, where the data already exist in some form as variables in the MATLAB workspace. `VFLUXFormat` is called first, in order to prepare the data structure for use with `VFLUX`. `VFLUX` is then called to perform the resampling, filtering, and flux calculation routines on the formatted data structure.

Descriptions of the inputs and options for the two functions are presented below, and can also be displayed in the MATLAB command window by typing `help vfluxformat` and `help vflux` at the command prompt.

3. Input Time Series Data

VFLUX is designed to process temperature time series measured by multiple temperature sensors in a single streambed vertical profile (Figure 1). Table 1 contains example time series data collected from a vertical temperature profile.

The time series data include a column of sample times, in days, and a column for each sensor of temperature values, in degrees Celsius, that were recorded at each sample time. The sample time values must be evenly-spaced in time and monotonically increasing throughout. For example, the data excerpted in Table 1 were sampled every 10 minutes without exception. Any gaps in time that occurred during data collection must be filled by interpolation. The time series data must be imported as array variables into the MATLAB workspace, either as individual column vectors or as a single large matrix.

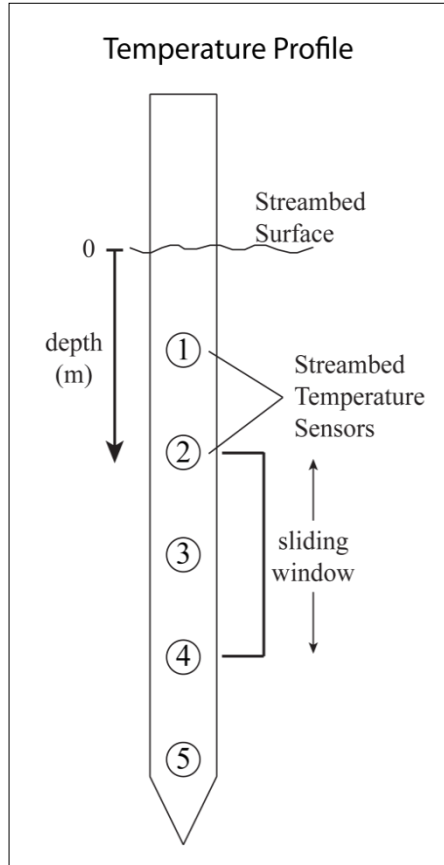


Table 1. An example of temperature time series data collected from a vertical temperature profile.

Time (days)	Temperature (degrees C)			
	Sensor 1	Sensor 2	Sensor 3	...
40052.50696	19.138	19.194	19.195	
40052.51390	19.326	19.382	19.320	
40052.52084	19.451	19.507	19.508	
40052.52779	19.639	19.633	19.633	
40052.53473	19.764	19.820	19.821	
40052.54168	19.951	20.008	19.946	...
40052.54862	20.077	20.133	20.134	
40052.55557	20.264	20.321	20.321	
40052.56251	20.390	20.446	20.446	
40052.56946	20.515	20.571	20.572	
...	...	...	...	

Figure 1. A typical vertical temperature profile, showing five temperature sensors installed in the streambed. The sensors are not necessarily evenly-spaced. The sliding window (in this case, two sensor-spacings long) identifies sensor pairs between which flux will be calculated.

In the event that different sensors were sampled on different time schedules or for different lengths of time, then there must be a separate time series for each of the schedules. For example, if sensors 1, 2 and 3 were sampled on schedule A and sensors 4 and 5 on schedule B, then there would be two time columns, A and B, the first associated with temperature columns for sensors 1, 2 and 3, and the second with temperature columns for sensors 4 and 5.

It is very important to note that VFLUX can only be used to analyze a raw dataset in which a strong periodic temperature signal exists (typically the diurnal, 24-hour signal driven by solar input) for a length of time long enough to filter the signal and discard the edge effects (see Flux Results section, below). This typically means at least one week of continuous time series data containing a 24-hour temperature signal.

4. Usage

A simple example of time series data from a vertical temperature profile is included as a mat-file in the “vflux\example” folder. Load the mat-file in order to import the time and temp variables into the MATLAB workspace. This example profile has six temperature sensors, located at depths of 0.05, 0.10, 0.15, 0.20, 0.25 and 0.30 meters below the streambed, all sampled at the same times. This data will be used in the following example commands.

4.1 Running VFLUXFormat

To prepare the time series data for analysis with VFLUX, first run VFLUXFormat (vfluxformat) to create a formatted data structure. The command syntax is:

```
data_struct = vfluxformat(time, temp, depth)
```

where `data_struct` is a name for the formatted data structure; `time` is the column of sample times; `temp` is a matrix of temperature columns (i.e., the temperature columns concatenated horizontally, such as in the notation `[temp1 temp2 temp3]`); and `depth` is a row vector of depth positions, in meters, for each of the sensors, where each column represents one sensor (corresponding to the columns in `temp`).

Example command:

```
site12 = vfluxformat(site12_data(:,1), site12_data(:,2:7), [0.05 0.10 0.15  
0.20 0.25 0.30]);
```

If different sensors were sampled on different time schedules or for different lengths of time, then `vfluxformat` resamples the input time series to the "lowest common denominator", that is, it trims the input series to the shortest time range that is common to all the input series, and interpolates/resamples the input series to have the lowest sampling rate of all the input series. Interpolation is performed with the MATLAB function `interp1`, using the linear method. In this case, construct the command with more than one triplet of `time`, `temp` and `depth` inputs, like in the following command:

```
data_struct = vfluxformat(timeA, tempA, depthA, timeB, tempB, depthB, ...)
```

Note that it is necessary that the order of temperature columns be in order of increasing sensor depth, throughout all the input matrices, so that the depth vectors concatenated together are monotonically increasing.

The output from the `vfluxformat` command is a MATLAB data structure (`data_struct`) that contains three arrays, or ‘fields’: `time`, a column of sample times in days; `temp`, a matrix of temperatures in degrees Celcius, where each column is data from a separate sensor; and `depth`, a row vector of depth positions in meters for each of the sensors, where each column represents one sensor. Fields in structures can be referenced using the “dot” notation: `data_struct.time`.

For more information, type `help vfluxformat` at the command prompt.

4.2 Running VFLUX 2

After the data structure is created, run VFLUX (`vflux`) on the structure to resample, filter, and calculate fluxes. The command syntax is:

```
data_struct = vflux(data_struct, rfactor, windows, Pf, n, beta, Kcal, Cscal,
                    Cwcal)
```

Example command:

```
site12 = vflux(site12, 0, [1 2], 1, 0.28, 0.001, 0.0045, 0.5, 1);
```

The inputs parameters are as follows:

`rfactor` : a positive integer factor by which to reduce the sampling rate. For example, if the original sampling rate is 72 samples/day, and the desired reduced sampling rate is 12 samples/day, then `rfactor`=72/12=6. *A reduced sampling rate of approximately 12 samples per fundamental cycle (see `Pf`, below) is recommended in order to eliminate spurious filtration artifacts.* If `rfactor`=1, then no resampling is performed. If `rfactor`=0, then `vflux` will calculate an appropriate `rfactor` so that the reduced sampling rate is close to 12 samples per fundamental cycle (as input as `Pf`).

`windows` : a scalar or vector of positive integers, where each element is a sliding sensor-spacing "window", in units of sensor-spacings, used to identify sensor pairs for flux calculations (see Figure 1). `vflux` will calculate flux between all the sensor pairs that are separated by the "window"-number of sensor-spacings. For example, if there are 5 sensors in the profile and `windows`=1, `vflux` will calculate fluxes between sensors 1 and 2, 2 and 3, 3 and 4, and 4 and 5 (the "window" is one sensor-spacing). If `windows`=[1 2], then `vflux` will calculate fluxes between the above pairs, plus between sensors 1 and 3, 2 and 4, and 3 and 5 (the "window" here is two spacings). By including 3 in the `windows` vector, sensor pairs 1 and 4 and 2 and 5 would also be included. If `windows`=4, then flux would only be calculated between sensors 1 and 5.

`Pf` : the period of the fundamental temperature signal to filter and use for flux calculations, in days (the typical value is 1, for the diurnal signal).

`n` : total porosity of the sediment, i.e., the volume of the pore spaces divided by the total volume (a typical value is 0.28).

`beta` : dispersivity, in meters (a typical value is 0.001).

`Kcal` : thermal conductivity, in cal/(s·cm·C) (a typical value is 0.0045).

`Cscal` : volumetric heat capacity of the sediment, in cal/(cm³·C) (typical value is 0.5).

`Cwcal` : volumetric heat capacity of the water, in cal/(cm³·C) (typical value is 1.0).

The above thermal parameters (`Kcal`, `Cscal`, and `Cwcal`) are defined in the units used by Lapham (1989), which describes how to estimate thermal parameters from sediment texture.

An optional input parameter, `'unattended'`, can be added to the end of the input command. The inclusion of this parameter causes VFLUX to run in unattended mode, in which all pauses, diagnostic plots, and user interaction with the program are suppressed. This can be useful if VFLUX is invoked from a custom MATLAB script or if it needs to be run many times in a row, for example, in a sensitivity analysis. In this case, the example command would be:

```
site12 = vflux(site12, 0, [1 2], 1, 0.28, 0.001, 0.0045, 0.5, 1,
'unattended');
```

For more information on inputs and outputs, type `help vflux` at the command prompt.

When executed, `vflux` will produce three diagnostic plots for each temperature series (each sensor), pausing after each. First, an autoregression spectrum from the Captain ARSPEC function is displayed, showing the time series in the frequency domain (Figure 2). Vertical red lines are shown at the fundamental signal period (as input for `Pf`), and at all harmonics down to the Nyquist period. If the time series contains the fundamental signal at sufficient power, there should appear peaks near the fundamental period and one or more of its harmonics.

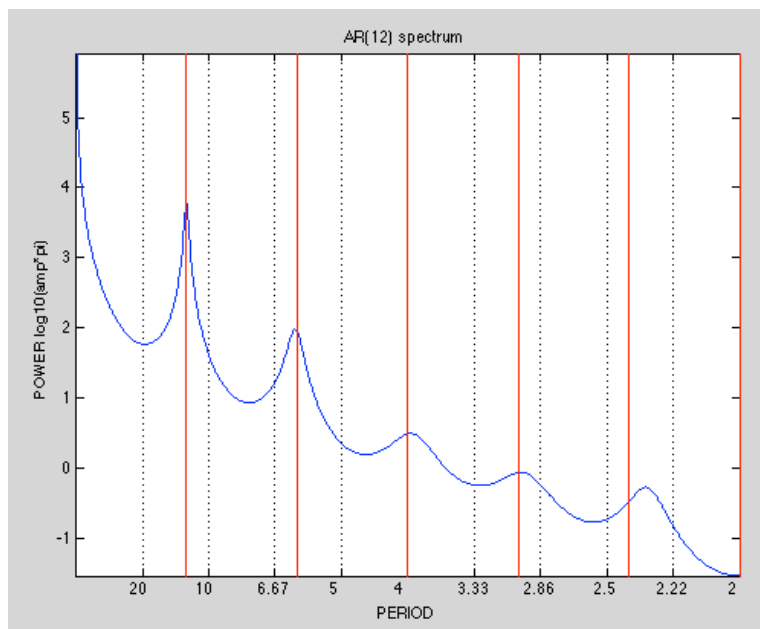


Figure 2. An example of the first diagnostic plot (created by ARSPEC), showing an AR spectrum with strong peaks at the fundamental period (12 samples/cycle) and the first two harmonics (6 and 4 samples/cycle).

Second, the DHROPT (DHR optimization) function tries to model the first few peaks of the autoregression spectrum (as indicated by the vector `P`, see *Additional Switches*, below), and it displays a plot of the actual and fitted spectrum (Figure 3). If the optimization is successful, the modeled red line should fit the actual blue line well over the fundamental peak and any other selected harmonics.

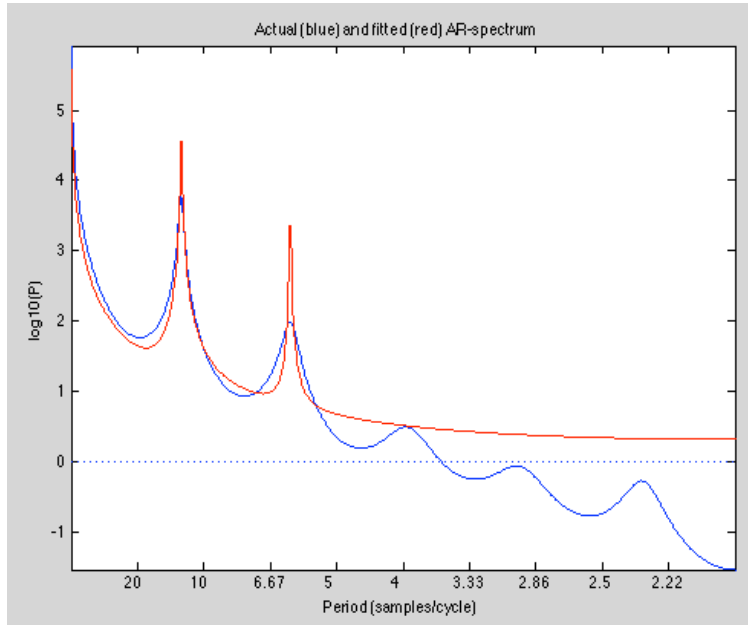


Figure 3. An example of the second diagnostic plot (created by DHROPT), showing the actual and modeled autoregression spectrum.

Third, VFLUX displays a plot of the filtering results, showing the actual time series data (“time series”) and the components of the time series identified by DHR (Figure 4). The “trend” is the part of the time series identified as non-cyclical. The “fundamental component” is the signal of interest (P_f), isolated from the trend and other components. The “fundamental amplitude” is the amplitude of the fundamental component at every time step. The harmonic components are any harmonics that were identified and filtered from the time series. The “fit to time series” is the complete DHR model of the data, or the sum of the trend, fundamental, and harmonic components. The “residual” is the error, or the difference between the actual data and the model fit.

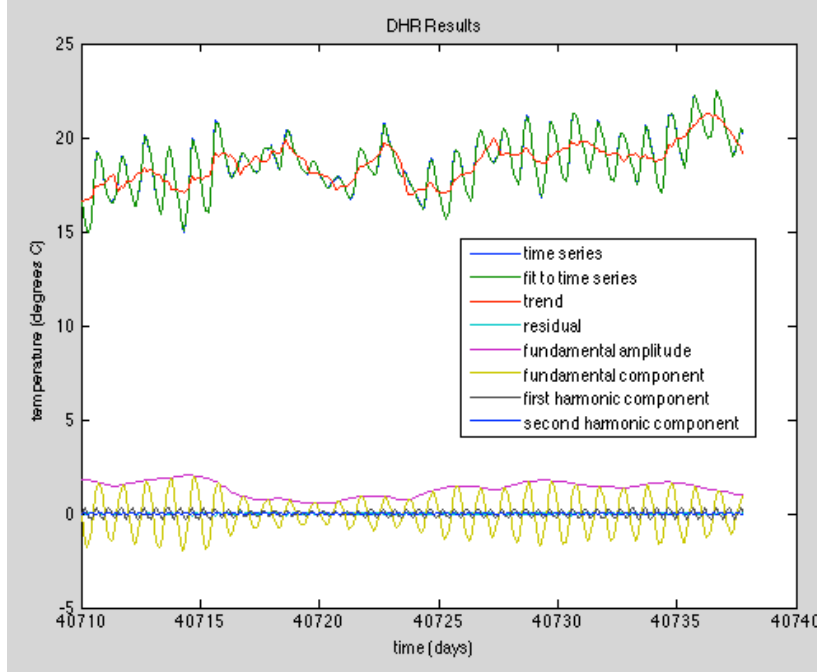


Figure 4. An example of the third diagnostic plot, showing the original time series, the modeled fit to the time series, the non-cyclical trend, the residual (time series minus fit), the fundamental component and its amplitude, and any other selected components (harmonics).

The output from the `vflux` command is described in the next section, Flux Results.

4.3 Post-calculation visualization routine

A post-calculation plotting routine can be run after VFLUX completes flux calculations. At the end of the VFLUX run, the program asks the user whether they would like to plot results. If ‘no’ is chosen, VFLUX ends. If ‘yes’ is chosen, the program displays plots of temperature versus time (for raw and filtered data sets), and amplitude and phase versus time (Figure 5). The program also prompts for the temperature sensor resolution. If entered, this value is displayed as a horizontal line on the time versus amplitude plot. The data from any time series that have amplitudes below the sensor resolution should be treated with suspicion. A second figure window is then displayed, with six plots of flux through time, one for each of the analytical methods (Figure 6). Finally, a third figure is displayed, with four plots; two show flux through time for the two “combined” methods (Luce et al., 2013; McCallum et al., 2012) and two show thermal diffusivity output from the same methods through time (Figure 7). The magnifying glass tools can be used on any of these plots.

The routine will then ask the user to display sensor spacing statistics. If chosen, it displays a series of bar charts, one chart for each sensor spacing window (Figure 7). Each chart shows a bar for a particular pair of sensors, labeled by the center-of-pair depth. The bar height represents the percentage of flux estimates through time that were successfully calculated for that sensor pair. An unsuccessful calculation is one that results in NaN (not-a-number) due to the amplitude ratio being ≥ 1 , the phase shift being ≤ 0 , or other calculation failures, most of which are caused by non-ideal sensor spacing.

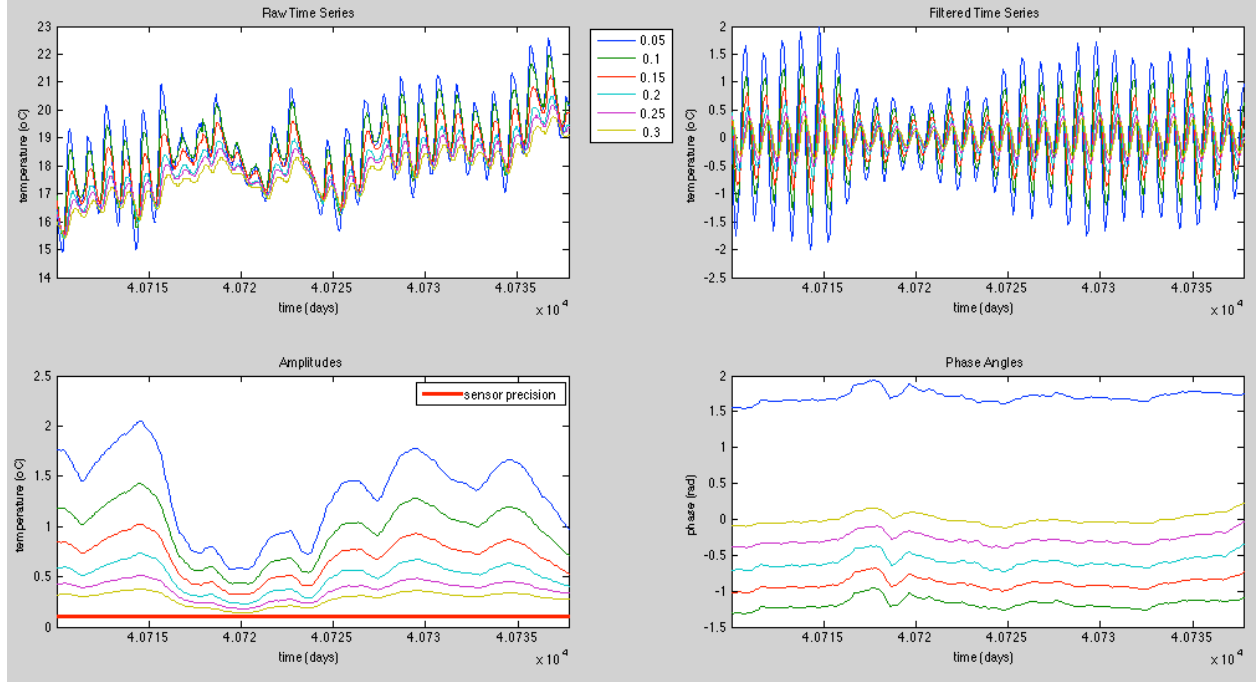


Figure 5. Plots of VFLUX results, including the raw time series, filtered time series, and sensor amplitudes and phase angles through time. The bold red line on the amplitude plot represents the temperature sensor precision in degrees C.

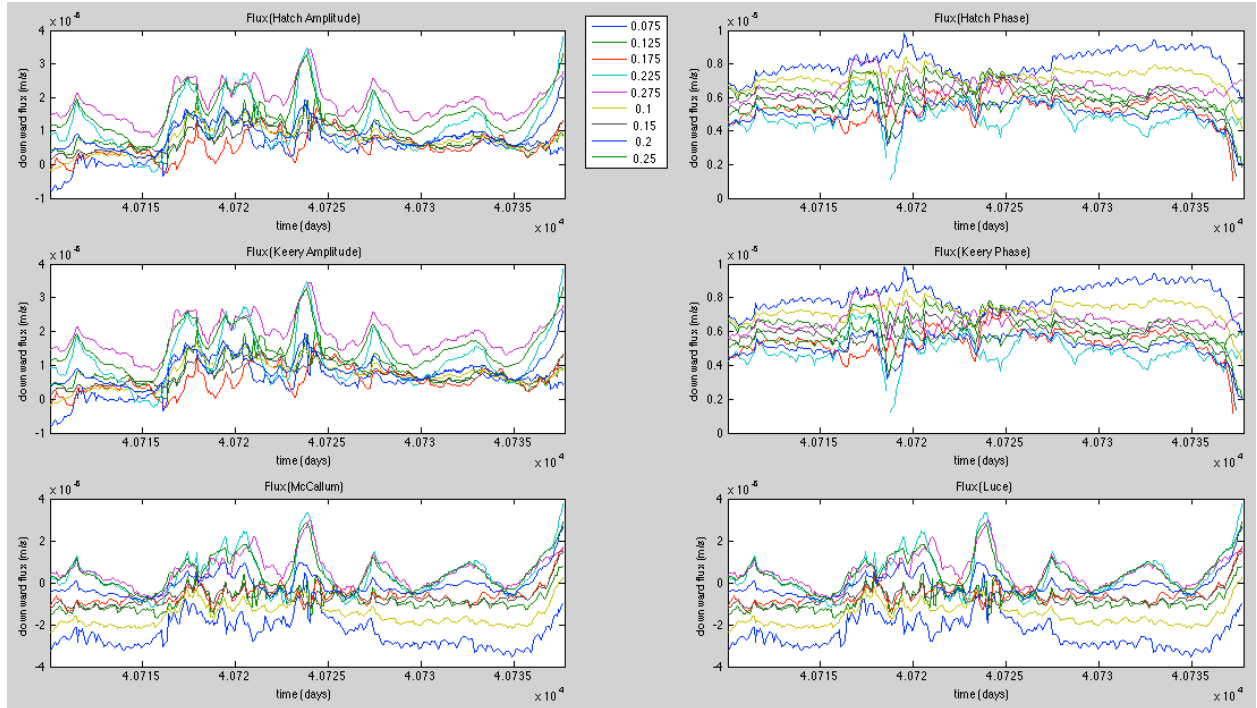


Figure 6. Plots of VFLUX results, showing flux through time calculated with each of the six analytical methods, for all sensor pairs (identified by their center-of-pair depth).

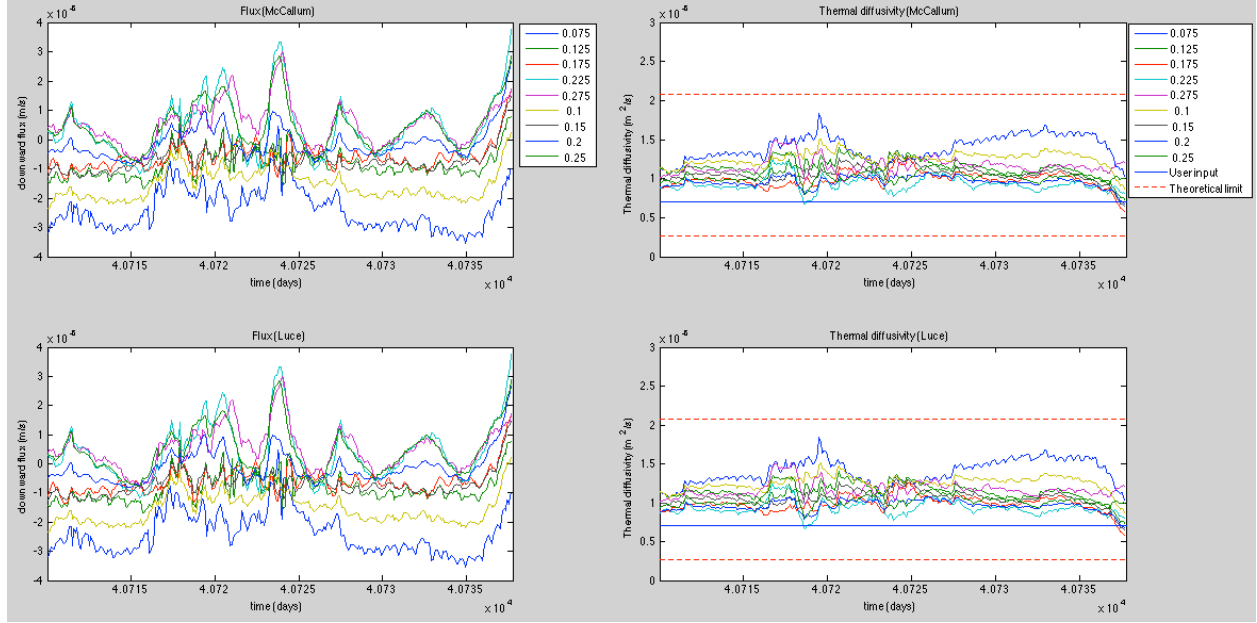


Figure 7. Plots of VFLUX results for the combined methods (Luce et al., 2013; McCallum et al., 2012), showing (on the left) flux through time calculated with the two combined methods, for all sensor pairs (identified by their center-of-pair depth) and (on the right) thermal diffusivity through time for each center-of-pair depth. On the plot of thermal diffusivity through time, the thermal diffusivity associated with the user input thermal parameters (“user input”) and the theoretical limits for thermal diffusivity are shown.

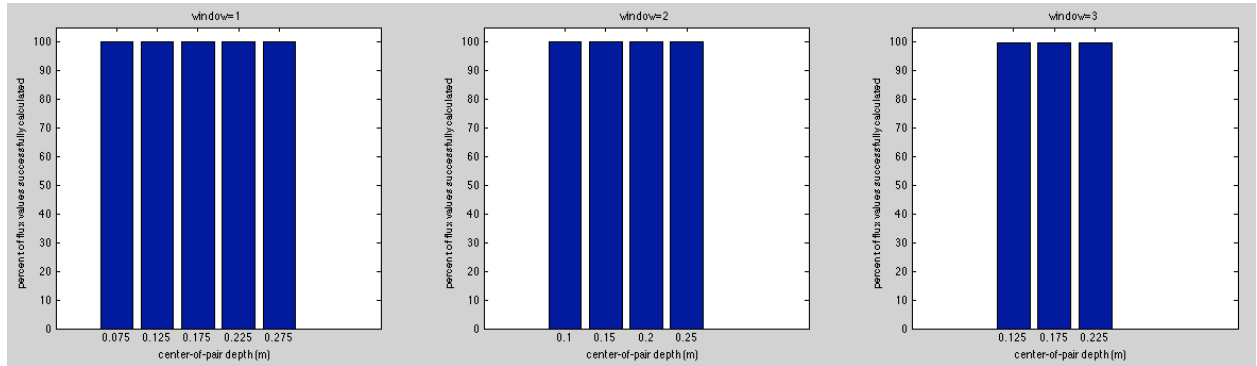


Figure 7. Bar charts showing the percent of flux values successfully calculated through time for each identified sensor pair.

4.4 Additional options from McCallum et al. (2012) and Luce et al. 2013 output (in *vflux_post.m*)

Additional functionality has been added in VFLUX 2 (in *vflux_post.m*), to refine the plausible range of thermal diffusivity that could be expected at a field site. This analysis can be useful to identify time periods where flux estimates may be unreliable. Large changes in thermal diffusivity with time typically occur when large changes in flux occur. Flux estimates during periods where the time series of predicted calculated thermal diffusivity values are outside of a plausible range that could be expected from porous media are likely to be erroneous.

In order to further utilize thermal diffusivity output, the user is prompted “Would you like to refine the range of plausible thermal diffusivity for the McCallum/ Luce plots?”. If ‘yes’ is chosen, the user can enter lower and upper expected values for porosity (–), density of the solid phase (kg/m^3), specific heat capacity of the solid phase ($\text{J}/(\text{kg}\cdot^\circ\text{C})$), and thermal conductivity of the solid phase ($\text{W}/(\text{m}\cdot^\circ\text{C})$). For each parameter, the user is prompted with typical values.

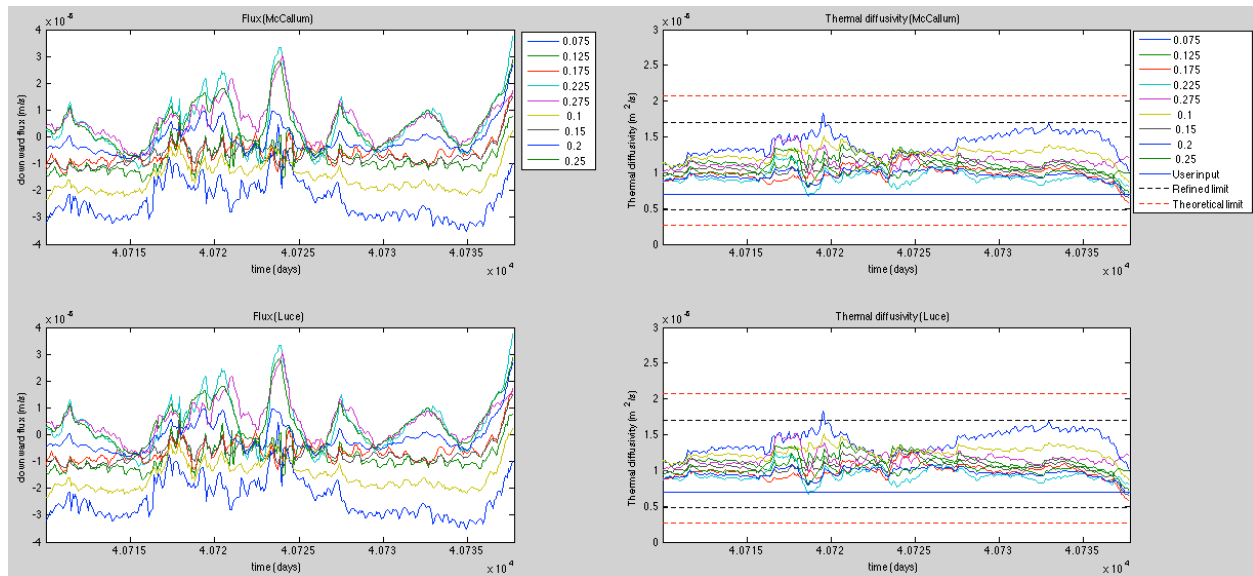


Figure 8. Time series of flux (left) and thermal diffusivity (right) from the Luce et al. 2013 and McCallum et al. 2012 methods, showing the refinement of the expected range of thermal diffusivity (black lines in right panels), as well as the wider, theoretically plausible range of values expected for thermal diffusivity.

4.5 Additional Switches to Control DHR Filtering (in `vflux_dhr.m`)

The method of filtering the fundamental frequency using Dynamic Harmonic Regression (DHR) is not trivial, and an understanding of DHR as implemented in the Captain Toolbox is recommended. See the Captain Toolbox documentation for more information. See Taylor et al. (2007) for a step-by-step example of DHR filtering with the Captain Toolbox.

Additional switches are included in the VFLUX program in order to control the way DHR is used with your data. These switches exist in the text of the m-file called “`vflux_dhr.m`”, and can be edited by typing `edit vflux_dhr` at the command prompt. Scroll down to the Additional Switches section, which starts at line 44.

`show_plots`: This switch shows or hides the diagnostic plots (described above). Set to 1 to pause after each plot, or 0 to continue without pausing (the plots won’t be visible). The default is 1. If `vflux.m` was invoked with the ‘`unattended`’ switch, then `show_plots` is automatically set to 0.

`ARorder` is a positive integer specifying the Auto-Regression spectrum order used in the Captain ARSPEC and DHROPT programs. Set to a positive integer (such as 12) to fix the AR order for all the time series. Set to 0 to choose the AR order automatically (using an AIC algorithm) individually for each time series. This setting may occasionally produce an AR order too low, in which case the fundamental frequency might not be identified, or too high, in which case spurious power peaks may be identified. Set to -1 to automatically set the AR order equal to the period of the fundamental component (in samples per cycle), rounded to the nearest integer. The default is -1. See the Captain Toolbox ARSPEC documentation for more.

`pstring`, unfortunately, is an awkward switch. `pstring`, the variable which is created here, is a text string that represents a complete MATLAB command, which will be executed when the program runs to create the variable `P`. The default line that creates `pstring` looks like:

```
pstring = 'P=[0 Pfs pks(3) pks(4)]' ; %comments
```

where everything within the purple quotes is the command to create `P`. `P` is an input to the DHR program that indicates the periods (in samples per cycle) of the trend and harmonic components to identify and filter using DHR. `P` is a row vector, where each element in the vector is a component to filter, in samples/cycle. In its simplest explicit form, an example of the `P` command might be `P=[0 12 6 4]`, where 0 represents the trend, 12 represents the fundamental component of 12 samples/cycle, and 6 and 4 are the first and second harmonics (6 and 4 samples/cycle). However, because VFLUX is designed to handle various fundamental frequencies, some shorthand is typically used to identify the peaks more generally. 0 will always mean the trend. `Pfs` can be used to refer to the fundamental component, as in `P=[0 Pfs Pfs/2 Pfs/3]`, which is equivalent to the explicitly-defined example above if the fundamental frequency is 12 samples/cycle. Or use `components` to refer collectively to the fundamental component and all harmonics down to the Nyquist frequency. If the fundamental component is 12 samples/cycle, then `P=[0 components]` is equivalent to `P=[0 12 6 4 3 2]`. `Pfs` is the same as `components(1)`. Finally, use `pks(x)` to indicate the x-th peak identified by the ARSPEC program. This is useful if the period of the components varies from one time series to the next, as is often the case. If the results from ARSPEC are clean and consistent without spurious peaks, then `pks(2)` is usually the fundamental, and `pks(3)` the first harmonic, etc. The default setting uses `pks` in `P=[0 Pfs pks(3) pks(4)]`, which identifies the trend, the fundamental, and the third and fourth peaks (usually the first and second harmonics). There are many example `pstring` commands in the m-file, all but one of which will be commented out by including a percent sign at the beginning of the line. Only *one* statement for `pstring` should be uncommented and active. In general, it is wise to include the trend, the fundamental, and at least one harmonic (if it is a significant component of your time series) in the command for `P`. It might be a good idea to explore your data with ARSPEC before committing to a particular `P` vector setting. See the Captain Toolbox ARSPEC, DHR, and DHROPT documentation for more.

`TVP` is a vector used by the DHROPT program, which determines the NVR model to use in optimization for the trend (first element of `TVP`) and harmonic components (second element of `TVP`). See the Captain Toolbox DHR and DHROPT documentation. 0=Random Walk, and 1=Integrated Random Walk. Typically, either `TVP=[1 0]` or `TVP=[0 0]`, the default, seem to work best. Use whichever combination best fits the actual data in the second diagnostic plot, created by DHROPT.

Remember, if the diagnostic plots suggest that the chosen parameters and switches are not ideal, it is possible to terminate the VFLUX program at any time by pressing CTRL-C.

5. Flux, Thermal Diffusivity and Sensor Spacing Results

After `vflux` runs successfully, it will create eleven new fields within the data structure `data_struct`. The first five are intermediate products of the data analysis:

`mtime` is the time vector downsampled by the factor `rfactor`. `mtime` will contain every `rfactor`-th value from `time`, starting with the first value.

`rtemp` is the resampled `temp` matrix, created using the MATLAB `resample` function (part of the Signal Processing Toolbox). Each row in `rtemp` corresponds to a time in `mtime`.

`amp` is the amplitude of the fundamental component at each time in `mtime`, in degrees C, in a matrix of the same size as `rtemp`.

`phs` is the phase of the fundamental component at each time in `mtime`, in radians, in a matrix of the same size as `rtemp`.

The last six output fields contain information about the flux calculations, the vertical flux rates themselves, and the program parameters. Note that all flux values, reported in units of meters per second in the downward direction, represent the true volumetric flux of fluid (volume per area per time), and *not* linear particle velocity (distance per time). In order to get linear particle velocity, it is necessary to divide the volumetric flux by the effective porosity of the sediment.

`fluxinfo` is a metadata matrix that contains information about the columns in the other flux result matrices (`fluxha`, etc.). Each column represents a sensor pair for which fluxes were calculated. For each column, the first row contains the window used (in units of sensor-spacings), the second row contains the depth of the upper sensor in meters, the third row contains the depth of the lower sensor in meters, and the fourth row contains the center-of-pair depth (the average of the upper and lower depths).

`fluxha` is a matrix of vertical flux rates in meters/second, calculated with the Hatch Amplitude method, at each time in `mtime`. Positive values are downward flux, and negative are upward. A value of NaN (not a number) means that flux could not be calculated at that time. Each column contains results from the sensor pair indicated in `fluxinfo`. The matrix has the same number of rows as `mtime`, and same number of columns as `fluxinfo`.

`fluxhp` is a matrix of vertical flux in meters/second, calculated with the Hatch Phase method, at each time in `mtime` and for each sensor pair in `fluxinfo`.

`fluxka` is a matrix of vertical flux in meters/second, calculated with the Keery Amplitude method, at each time in `mtime` and for each sensor pair in `fluxinfo`.

`fluxkp` is a matrix of vertical flux in meters/second, calculated with the Keery Phase method, at each time in `mtime` and for each sensor pair in `fluxinfo`.

`fluxm` is a matrix of vertical flux in meters/second, calculated with the McCallum combined Amplitude and Phase method, at each time in `dttime` and for each sensor pair in `fluxinfo`.

`kem` is a matrix of thermal diffusivity in meters²/second, calculated with the McCallum combined Amplitude and Phase method, at each time in `dttime` and for each sensor pair in `fluxinfo`.

`fluxl` is a matrix of vertical flux in meters/second, calculated with the Luce combined Amplitude and Phase method, at each time in `dttime` and for each sensor pair in `fluxinfo`.

`kell` is a matrix of thermal diffusivity in meters²/second, calculated with the Luce combined Amplitude and Phase method, at each time in `dttime` and for each sensor pair in `fluxinfo`.

`dzl` is a matrix of sensor spacing in meters, calculated with the Luce combined Amplitude and Phase method, at each time in `dttime` and for each sensor pair in `fluxinfo`. This output can be used to determine streambed scour. There is currently no plotting option to display `dzl` output.

`parameters` is a cell array that records the input parameters and other program settings that were used to run `vflux`. The first row of `parameters` is the name of the parameter, the second row is the input variable that represents the parameter, the third row is the units of the parameter, and the fourth row is the value of the parameter. Some cells (such as in the units row) are left empty by design.

Plots similar to those displayed by the post-calculation visualization routine at the end of a VFLUX run can also be created on the MATLAB command line after VFLUX has exited. For example, the command `plot(site12.time, site12.temp)` plots all of the raw temperature time series as lines on a single plot. `plot(site12.dtime, site12.ftemp)` plots the filtered temperatures through time. `plot(site12.dtime, site12.amp)` plots amplitude versus time for all sensors. `plot(site12.dtime, site12.amp(:,4))` plots the amplitude of only the fourth temperature sensor through time (fourth column in the “amp” matrix). `plot(site12.dtime, site12.fluxha, '.')` plots Hatch Amplitude flux through time as point symbols. `plot(site12.dtime, site12.fluxkp(:,2), '.')` plots Keery Phase flux through time for only the second sensor pair (second column in “fluxkp” matrix).

It is recommended that the first 2 or 3 days of flux data from the beginning and end of the data collection periods be discarded, due to the edge effects of digital filtering (Hatch et al. 2006; Keery et al. 2007), although that choice is left up to the user and is not implemented in VFLUX.

6. Known Issues

(1) For users who have access to the MATLAB Statistics Toolbox, improvements can be made to the Monte Carlo error analysis script (`vfluxmc.m`) by utilizing the `nanmean` and `nanstd` functions. With the use of the `mean` and `std` functions, a NaN value from a single realization can prevent VFLUX from producing estimates of the upper and lower 95% confidence limits of flux (e.g. `fluxhahigh` and `fluxhalow`).

To make the update in the `vfluxmc.m` script (which is only required once), search for the comment `% Calculate flux distributions`. Immediately below this comment, there are 6 instances

of mean (change to nanmean, e.g. change first instance to `=nanmean(output.fluxhamc,3)`), and six instances of std (change to nanstd). These changes will allow estimates of upper and lower 95% confidence limits even if a NaN value occurs.

Please send bug reports to Ryan Gordon at rpgordon@syr.edu or Laura Lautz at llautz@syr.edu.

7. Authors

VFLUX was created by Ryan P. Gordon^{1*}, Jeff M. McKenzie², Laura K. Lautz¹, and Martin A. Briggs¹.

Updates to VFLUX (VFLUX 2) by Dylan Irvine^{3**}, Laura K. Lautz¹, Martin A. Briggs¹, Ryan P. Gordon^{1*} and Jeff M. McKenzie².

¹Syracuse University, Department of Earth Sciences, 204 Heroy Geology Lab, Syracuse, NY 13244, USA.

²McGill University, Earth and Planetary Sciences, 3450 University Street, Montreal, Quebec, H3A 2A7, Canada.

³Monash University, School of Earth, Atmosphere and Environment, Clayton, Vic, Australia.

*Contact e-mail address: rpgordon@syr.edu

**Contact e-mail address: dylan.irvine@monash.edu

8. Copyright and License

This documentation is copyright (c) 2011, 2012 Ryan P. Gordon. All rights reserved.

VFLUX software (including the following computer files: `vflux.m`, `vflux_dhr.m`, `vflux_flux.m`, `vflux_resample.m`, `vflux_post.m`, `vfluxformat.m`, `vfluxmc.m`, and `vfluxsens.m`) are distributed under the following Simplified BSD License:

Copyright (c) 2011, Ryan P. Gordon.
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

- (1) Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
- (2) Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

9. References

- Gordon, R. P., L. K. Lautz, M. A. Briggs, and J. M. McKenzie. 2012. Automated calculation of vertical pore-water flux from field temperature time series using the VFLUX method and computer program. *J. Hydrol.* 420–421: 142-158.
- Hatch, C. E., A. T. Fisher, J. S. Revenaugh, J. Constantz, and C. Ruehl. 2006. Quantifying surface water-groundwater interactions using time series analysis of streambed thermal records: Method development. *Water Resour. Res.* 42: W10410.
- Irvine, D.J., L. K. Lautz, M. A. Briggs, R. P. Gordon, and J. M. McKenzie. in review. Experimental evaluation of the applicability of phase, amplitude, and combined methods to determine water flux rates from temperature time series using VFLUX 2. Submitted to *J. Hydrol.*
- Keery, J., A. Binley, N. Crook, and J. W. N. Smith. 2007. Temporal and spatial variability of groundwater-surface water fluxes: Development and application of an analytical method using temperature time series. *J. Hydrol.* 336: 1-16.
- Lapham, W. W. 1989. Use of temperature profiles beneath streams to determine rates of vertical ground-water flow and vertical hydraulic conductivity. U.S. Geol. Surv. Water Supply Pap. 2337.
- Luce, C. H., D. Tonina, F. Garigilo, and R. Applebee. 2013. Solutions for the diurnally forced advection-diffusion equation to estimate bulk fluid velocity and diffusivity in streambeds from temperature time series. *Water Resour. Res.* 49: 488-506..
- McCallum, A. M., M. S. Andersen, G. C. Rau, and R. I. Acworth. 2012. A 1-D analytical method for estimating surface water-groundwater interactions and effective thermal diffusivity using temperature time series. *Water Resour. Res.* 49: W11532.
- Taylor, C. J., D. J. Pedregal, P. C. Young, and W. Tych. 2007. Environmental time series analysis and forecasting with the Captain toolbox. *Environ. Modell. Softw.* 22: 797-814.
- Young, P. C., D. J. Pedregal, and W. Tych. 1999. Dynamic harmonic regression. *J. Forecast.* 18: 369-394.
- Young, P. C., C. J. Taylor, W. Tych, D. J. Pegregal, and P. G. McKenna. 2010. The Captain Toolbox. Centre for Research on Environmental Systems and Statistics, Lancaster University, UK. Internet: <http://www.es.lancs.ac.uk/cres/captain>.